Ders 20 Genişletilmiş Notu: Insertion Sort, Histogram, Sezar Şifreleme ve İki Sıralı Diziyi Birleştirme

Ders 20 Genişletilmiş Notu: Insertion Sort, Histogram, Sezar Şifreleme ve İki Sıralı Diziyi Birleştirme

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Insertion Sort'un Akış Diyagramı
- 2. Insertion Sort'un Avantajı
- 3. Büyük Diziler İçin Sıralama
- 4. Histogram Problemi ve Sayaç Dizisi
- 5. Kaydırmalı Şifreleme ve Modüler Aritmetik
- 6. Mod mu, If mi?
- 7. İki Sıralı Diziyi Birleştirme

1. Insertion Sort'un Akış Diyagramı

Hoca derse, uygulama dersinde yarım kalan insertion sort ile başladı. Bu konu bitince sıralama algoritmalarıyla ilgili söylenecekler tamamlanmış olacaktı.

> **Hocanın vurgusu:** Sıralama algoritmalarını iyi öğrenmek önemlidir. Bundan sonraki süreçte hep karşınıza çıkacakları için onları tekrar tekrar kendi başınıza çözebiliyor olmanız gerekir.

Insertion sort'un mantığı kart dizmeye benzer. Bir öğrenci algoritmayı şöyle özetledi: İkinci elemandan başlanır. Eleman, sol tarafındakiler arasında olması gereken yere konur. Üçüncü, dördüncü ve beşinci elemanlar için de aynısı yapılır. Dış döngü ikinci elemandan başlar ve N'e kadar gider, çünkü son eleman da yerine konmalıdır.

İç döngü için neden while kullanılır? Dış döngünün adım sayısı bellidir, ama her turda iç döngüde farklı sayıda kontrol yapılır. Bir eleman hiç hareket etmeyebilir ya da en başa kadar gidebilir. Adım sayısı belli olmadığı için while uygundur.

1.1 İlk Çözüm: Yan Yana Elemanları Değiştirmek

Bir öğrencinin ilk çözümü şuydu: `J = I` yapılır. `J > 1` ve `Dizi[J] < Dizi[J-1]` olduğu sürece `Dizi[J]` ile `Dizi[J-1]` geçici bir değişken yardımıyla yer değiştirir, sonra J bir azaltılır.

Hoca bu çözümü `5 3 2 1 4 7 8` dizisi üzerinde özetledi. 3 alınır ve 5 ile yer değiştirir. Sonra 2 alınır, önce 5 ile, sonra 3 ile yer değiştirir. Çözüm sıralar, ama her adımda iki eleman değiştirilir. Hocanın sorusu buydu: Her seferinde ikili eleman değiştirmek yerine bunu daha verimli yapabilir miyiz?

1.2 Yer Değiştirmek Yerine Kaydırmak

Bir öğrenci, elemandan küçük bir değer bulunana kadar ilerleyip yeri bulmayı ve elemanı oraya koymayı önerdi. Fikir doğruydu, ama geriye bir soru kalıyordu: 5, 3 ve 2 kendi yeni yerlerine ne zaman yazılacak?

Soru: While'dan çıktıktan sonra bir for döngüsüyle kaydırma yapılabilir mi? **Cevap:** Yapılabilir, ama bu fazladan bir döngü gerektirir. While'ın içindeyken bu iş halledilebilir.

> **Hocanın vurgusu:** Burası önemlidir. Her turda elemanlar bir sağa kaydırılır. Elimizdeki eleman küçük olduğu sürece örneğin 5, 1'in üzerine, 3, 5'in üzerine, 2 de 3'ün üzerine yazılır.

Böylece while'ın gövdesi iki satıra indi: `Dizi[J] = Dizi[J-1]` ve `J = J - 1`. Elimizdeki eleman ise while'dan çıkınca yerine yazılacaktı.

Bu noktada iki hata daha ortaya çıktı. Tahtadaki ilk hâlde döngüden çıkınca `Dizi[J-1]` gözüne yazılıyordu. Bir öğrenci doğrusunun `Dizi[J]` olduğunu söyledi. Hoca bunu diziyle deneyerek görmeyi önerdi.

Hoca denemeye geçmeden önce başka bir öğrenci bir tutarsızlığa dikkat çekti. Dizi, akış diyagramının başında `A` adıyla okunmuş, sonraki bütün adımlarda `Dizi` diye kullanılmıştı.

> **Hocanın vurgusu:** Burası çok önemlidir. Girdi `A` olarak alınıp bütün işlemler `Dizi` adıyla yapılırsa okuyan karışır. Değişken adlarında tutarlı olmak gerekir.

1.3 TMP'ye Neden İhtiyaç Var?

Hoca, o andaki diyagramı `5 3 2 1 4 7 8` dizisiyle adım adım denedi. O sırada while koşulu `Dizi[I] < Dizi[J-1]` idi ve döngüden çıkınca `Dizi[J] = Dizi[I]` yapılıyordu.

- $I = 2$, $J = 2$ olur. 2, 1'den büyüktür ve `Dizi[2] = 3`, `Dizi[1] = 5`'ten küçüktür.
- 5 ikinci göze yazılır. Dizi `5 5 2 1 4 7 8` olur.
- $J = 1$ olur. 1, 1'den büyük olmadığı için döngüden çıkılır.
- `Dizi[J] = Dizi[I]` çalışır. Ama `Dizi[2]` artık 5'tir. Birinci göze yine 5 yazılır ve 3 kaybolmuştur.

Bir öğrenci çözümü söyledi: `Dizi[I]`'nın ilk değeri bir değişkende tutulmalıdır. Hoca bunun için `TMP = Dizi[I]` adımını ekledi ve döngüden çıkınca `Dizi[J] = TMP` yazdı.

Bir süre sonra bir öğrenci, while koşulunda da `Dizi[I]` yerine `TMP` yazılması gerektiğini söyledi. Hoca soruyu sınıfa açtı.

Soru: TMP ile `Dizi[I]` aynı değerde olduğu için koşulda hangisini yazdığımız fark etmez mi? **Cevap:** Fark eder. İlk kaydırmada, örneğin 5, `Dizi[I]`'nın bulunduğu göze yazılır. I aynı kalsa da o gözdeki değer kaybolmuştur. Bu yüzden koşuldaki değişken TMP olmalıdır.

Soru: While'a ikinci kez girerken `Dizi[J]` ile `Dizi[J-1]` karşılaştırılmıyor mu? **Cevap:** Hayır. J ile J-1 gözleri değil, elimizde kalan eleman ile J-1 gözü karşılaştırılır. Baştaki elemanın diğer elemanlardan küçük olup olmadığına bakıyoruz. Bu yüzden `TMP < Dizi[J-1]` olduğu sürece döngü devam etmelidir.

Soru: Tahtadaki koşulda `Dizi[I]` yazıyordu. Analiz tesadüfen mi doğru çıktı? **Cevap:** Analiz TMP üzerinden yapıldı, ama diyagrama `Dizi[I]` olarak yazılmış kaldı. 3'ün üzerine 5 yazılınca karşılaştırma artık 5 ile yapılır, oysa karşılaştırılması gereken değer 3'tür. Diyagram `Dizi[I]` ile adım adım izlenirse hatalı olduğu görülür.

1.4 Son Akış Diyagramı

Tartışmaların sonunda algoritma şu adımlara oturdu:

1. Başla.
2. N ve `Dizi[N]` okunur.
3. Dış döngü: `I ← 2 || 1 || N`.
4. `J = I` ve `TMP = Dizi[I]` yapılır.
5. `J > 1` ve `TMP < Dizi[J-1]` olduğu sürece:

- `Dizi[J] = Dizi[J-1]`
- `J = J - 1`

6. While'dan çıkınca `Dizi[J] = TMP` yazılır.
7. Dış döngü bitince dur.

1.5 Örnek: `5 3 2 1 4 7 8`

$I = 2$ turunda $TMP = 3$ olur. 3, 5'ten küçüktür, 5 bir sağa kayar. $J = 1$ olunca döngüden çıkılır ve 3 birinci göze yazılır. Dizi `3 5 2 1 4 7 8` olur.

$I = 3$ turunda $J = 3$ ve $TMP = 2$ olur. 2, 5'ten küçüktür ve 5 kayar. $J = 2$ olur. 2, 3'ten de küçüktür ve 3 kayar. $J = 1$ olunca döngüden çıkılır. 2 birinci göze yazılır ve dizi `2 3 5 1 4 7 8` olur.

$I = 4$ turunda $TMP = 1$ olur. 1 sırayla 5, 3 ve 2'den küçüktür. Üçü de birer sağa kayar. TMP birinci göze yazılır ve dizi `1 2 3 5 4 7 8` olur.

Hoca kalan turları atlayıp sonda 8'in bir öncekinden küçük olmadığını söyledi. Bütün turlar aşağıdaki tablodadır. (ara adım eklendi)

I	TMP	Karşılaştırma	Kaydırma	Tur sonunda dizi
2	3	1	1	3 5 2 1 4 7 8
3	2	2	2	2 3 5 1 4 7 8
4	1	3	3	1 2 3 5 4 7 8
5	4	2	1	1 2 3 4 5 7 8
6	7	1	0	1 2 3 4 5 7 8
7	8	1	0	1 2 3 4 5 7 8

Tablodaki karşılaştırma sütunu `TMP < Dizi[J-1]` sorusunu sayar ve toplam 10'dur.

1.6 Kaydırma Olmadan Olur mu?

Soru: While içinde kaydırma yapmadan, I her arttığında yalnızca yan yana iki elemanın yerini değiştirsek olmaz mı? Sonuçta önceki elemanlar zaten yerine oturmuş olmuyor mu? **Cevap:** Doğru çalışmaz. Kodunu yazıp denerseniz görürsünüz. Örneğin `2 3 5` sıralı kısmından sonra 1 geldiğini düşünün. 1'in en başa kadar gitmesi gerekir. Bunun yolu 5'in 1'in yerine, 3'ün 5'in yerine, 2'nin de 3'ün yerine geçmesidir. Önceki elemanlar kendi içlerinde sıralıdır, ama yeni eleman onlardan küçük olduğu için onların da kayması gerekir.

> **Ders sonrası not:** Hoca bu örnekte "1, 5'ten büyükse" dedi; doğrusu "1, 5'ten küçükse"dir. Aynı cümlenin devamında 1'in en başa gitmesi gerektiği söylenir.

1.7 C Kodu

Hoca kod yazmadı. Aşağıdaki kod, son akış diyagramının C karşılığıdır. C'de dizi 0'dan başladığı için indisler bir azaltılmıştır: dış döngü 1'den başlar, iç koşul `j > 0` olur.

```c
#include <stdio.h>
#define MAX 100

int main(void)
{
    int dizi[MAX] = {5, 3, 2, 1, 4, 7, 8};
    int N = 7;
    int i, j, tmp;

    for (i = 1; i < N; i++) {
        tmp = dizi[i];
        j = i;
        while (j > 0 && tmp < dizi[j - 1]) {
```

```
            dizi[j] = dizi[j - 1];
            j = j - 1;
        }
        dizi[j] = tmp;
    }
    for (i = 0; i < N; i++)
        printf("%d ", dizi[i]);
    printf("\n");
    return 0;
}
```

Kod `gcc -Wall` ile uyarısız derlenmiş ve `1 2 3 4 5 7 8` çıktısını vermiştir.

2. Insertion Sort'un Avantajı

Hoca insertion sort'un avantajını sordu. Bir öğrenci, kaydırma sayesinde diğerlerine göre daha az karşılaştırma yapıldığını söyledi. Hoca bunu genel olarak söyleyemeyeceğimizi, karşılaştırmanın while'da yine yapıldığını belirtti. Başka bir öğrenci, en iyi durumlarda belki daha az karşılaştırma yapılabileceğini söyledi. Hoca, bilgisayarda "belki" diye bir durum pek olmadığını söyleyip bir örnek açtı.

2.1 En İyi Durum: `1 3 5 7 9`

Küçükten büyüğe sıralamak istediğimiz dizi zaten sıralıysa bu, en iyi durum senaryosudur:

- 3, 1'den küçük mü? Değil. İç döngüye girilmez.
- 5, 3'ten küçük mü? Değil. İç döngüye girilmez.
- 7, 5'ten küçük mü? Değil. İç döngüye girilmez.
- 9, 7'den küçük mü? Değil. İç döngüye girilmez.

Bütün elemanlar yerindedir ve iş biter. Burada $N = 5$ ve toplam karşılaştırma sayısı 4'tür.

Soru: En iyi durumda karşılaştırma sayısı sıfır mıdır, çünkü while'a hiç girilmiyor? **Cevap:** Hayır. While'a girmememizin sebebi ikinci koşuldur. Yani aslında bir karşılaştırma yapıyoruz ve onu da saymalıyız. Toplam $N - 1$ karşılaştırma yapılır.

Soru: For döngüsündeki kontrol de karşılaştırma sayılmıyor mu? **Cevap:** For döngüsü $N - 1$ kez döner, bu da yapılacak işin $N - 1$ tur olduğunu söyler. Ama asıl karşılaştırma, her turda bir kez sorulan "eleman `Dizi[J-1]`'den küçük mü?" sorusudur. Burada saydığımız odur.

2.2 Sıralıya Yakın Diziler

Hoca ardından sıralı olmayan ama sıralıya yakın bir dizi gösterdi. Bu dizide yalnızca son bölümde değişiklik yapılır, geri kalana dokunulmaz.

Hocanın sözlü verdiği `1 3 9 7 5` dizisi için turlar şöyledir (ara adım eklendi):

I	TMP	Karşılaştırma	Kaydırma	Tur sonunda dizi
2	3	1	0	1 3 9 7 5
3	9	1	0	1 3 9 7 5
4	7	2	1	1 3 7 9 5
5	5	3	2	1 3 5 7 9

İlk iki eleman hiç hareket etmez. Değişiklik yalnızca son bölümde olur.

Eleman olması gereken yerindeyse insertion sort onu hareket ettirmez ve onunla ilgili başka bir şey yapmaz. Bu yüzden algoritma sıralı ve sıralıya yakın diziler için oldukça avantajlıdır. Önemli olan, veri setinin çok karışık olmadığını, rastgele dağılmadığını bilmektir.

Hoca böylece üç sıralama türünün görüldüğünü hatırlattı: bubble sort, selection sort ve insertion sort.

3. Büyük Diziler İçin Sıralama

Soru: Milyonlarca elemanlı çok büyük bir dizide belli bir sayının altındaki değerleri bulmak istiyorum. Doğrudan arayıp başka bir diziye atmak mı, yoksa insertion sort ile sıralayıp belli bir noktadan aşağısını kullanmak mı daha avantajlıdır? **Cevap:** Önce dizinin genel olarak sıralı olup olmadığını bilip bilmediğimize bakılır. Hiçbir fikrimiz yoksa bu tür büyük diziler için en uygunu quick sort ya da merge sort kullanmaktır. Hoca, sorudaki doğrudan arama seçeneğini ayrıca değerlendirmedi.

Insertion sort en iyi durumda $N - 1$ adım yapar. Merge sort'un en kötü durumdaki karmaşıklığı ise $N \log N$'dir ve bu işi oldukça hızlandırır. Hoca bunu söylerken sözünü düzeltti ve bu değerin quick sort için değil, merge sort için geçerli olduğunu belirtti.

> **Ek bilgi (derste anlatılmadı):** Quick sort'un ortalama karmaşıklığı büyük olasılıkla $N \log N$, en kötü durumu ise N^2'dir. Hocanın bu ayrımı yapması büyük olasılıkla bundandır.

Merge sort'un bir dezavantajı vardır: bellek karmaşıklığı (space complexity). Eleman sayısı kadar ek yer gerekir. Hoca ayrıca ileride görülecek heap sort'tan söz etti. Onun karmaşıklığı da $N \log N$'dir. Bu algoritmaların birbirine göre küçük artıları ve eksileri vardır. Hoca, merge sort, quick sort ve heap sort'un büyük diziler için kullanıldığını ve böl ve yönet (divide and conquer) yaklaşımıyla çözdüğünü söyledi.

> **Ders sonrası not:** Hoca üçünün de böl ve yönet yaklaşımını kullandığını söyledi; merge sort ve quick sort için bu büyük olasılıkla doğrudur, heap sort ise büyük olasılıkla bu sınıfa girmez, yığın (heap) veri yapısına dayanır.

4. Histogram Problemi ve Sayaç Dizisi

Hoca yeni bir örnek verdi:

> 8 bit gri seviyesindeki bir resmin bilgileri verilmektedir. Resmin histogramını çıkaran algoritmanın akış diyagramını çiziniz.

Hoca burada genel bir hatırlatma yaptı. Akış diyagramlarının kodlarını derste yazmıyor, öğrencilerin yazdığını varsayıyordu.

> **Hocanın vurgusu:** Yaptığımız algoritmaların kodlarını kendiniz yazmanız önemlidir. Kod konusunda kendinizi böyle geliştirirsiniz. Sorun yaşarsanız uygulama saatine taşıyın, orada tartışalım.

4.1 Histogram Nedir?

Histogram almak yalnızca resimlere özgü değildir. Hoca soruyu önce metin üzerinden sordu: Bir metindeki karakterlerin histogramını çıkarın denirse ne anlarsınız? Bir öğrenci kelime başına tekrar sayısını önerdi. Hoca bunu harf olarak yorumlamamızı istedi: Metindeki her harften kaç adet olduğu bilgisi. Yani A'dan kaç tane var, B'den kaç tane var.

Resim için de aynısı geçerlidir. Bir öğrenci her satırdaki siyah ve beyazları önerdi, hoca bunun satırla ilgili olmadığını söyledi. İstenen, resimdeki renklerin sayısını veren algoritmadır: siyahtan bu kadar, grinin şu tonundan şu kadar. Başka bir deyişle farklı renklerin sıklığı bulunup ekrana yazdırılır.

4.2 Gri Seviye Ne Demektir?

Bir öğrenci 8 bit gri seviyesinin 8 farklı gri anlamına geldiğini söyledi. Başka bir öğrenci doğrusunu tarif etti: Siyahtan beyaza kadar gri tonlamalı bir resim. Hoca sınıfa siyahın ve beyazın değerini sordu ve RGB renklerini konuşmadığımızı özellikle belirtti.

Siyahın değeri 0'dır. Bir öğrenci beyaz için 1 dedi. Hoca sordu: 1 olsaydı kaç bit gri seviyesi derdim? Cevap 1 bittir. Hocanın yaptığı karşılaştırma şöyledir:

Bit sayısı	Siyah	Beyaz
1 bit	0	1
2 bit	0	3
8 bit	0	255

Bit seviyesi resmin ne kadar detaylı ifade edildiğini gösterir. Bit seviyesi yükseldikçe resmin renk derinliği artar. Burada renksiz bir resimden, gri tonları bulunan bir resimden söz ediyoruz. Gri tonlarını, ışığın o noktaya yansıması ya da o noktada bulunma durumu belirler. Renk konusuna daha sonra girilecektir.

Demek ki matriste 0 ile 255 arasında değerler vardır. Algoritmaya verilenler de bellidir: Matrisin satır ve sütun sayısı ile matristeki renk bilgileri dışarıdan okunur. Sonra farklı renklerin matriste kaç kez geçtiği bulunur.

4.3 Neden 256 Ayrı Değişken Olmaz?

Bir öğrenci, for döngüsü içinde her K değeri için ayrı bir sayaç değişkeni tanımladığını, böylece yaklaşık 256 değişkeni olduğunu söyledi.

Soru: For döngüsünde 0'dan 255'e kadar ilerleyerek 256 değişken tanımlanamaz mı? **Cevap:** Tanımlanamaz. `K0`, `K1`, `K2` gibi değişkenleri for döngüsü içinde o şekilde kullanma şansınız yoktur. Elle tek tek tanımlamak mümkündür, ama çok zahmetlidir.

Öğrencinin tespiti yine de doğruydu: Renklerin sayısını saklamak için 256 ayrı sayaca ihtiyaç var. Başka bir öğrenci çözümü önerdi: Bütün elemanları sıfır olan bir dizi tanımlanır. Örneğin 15 değeri görülünce dizinin 15. gözü bir artırılır. Bunun önünde bir engel yoktur.

Bu fikrin adı sayaç dizisidir (counter array). Kelimedeki harfleri sayarken bir sayaç değişkeni oluşturuyor, onu sıfırlıyor ve null görene kadar artırıyorduk. Burada aynı mantık, birden fazla sayaca ihtiyaç olduğunda dizi ile uygulanır.

> **Hocanın vurgusu:** Birden fazla sayaca ihtiyaç varsa, sayı 2'den fazla olsa bile, mantıklı olan bu sistemi dizi ile kurmaktır. Sayaç dizisini tanımladıktan sonra her gözünü mutlaka sıfırlamalısınız. Bu çok önemlidir, yoksa algoritma hatalı çalışır.

Çalışma mantığı şöyledir: 0 görülünce sıfırıncı göz bir artırılır. 5 görülünce beşinci göz bir artırılır. Bir 0 daha görülünce sıfırıncı göz kaldığı yerden artırılır. Sonunda başka 0 yoksa sıfırıncı gözde 2 yazar.

4.4 Akış Diyagramı

1. Başla.
2. N, M ve `resim[N][M]` okunur.
3. `I ← 0 || 1 || 255` için `counter[I] = 0` yapılır.
4. `I ← 0 || 1 || N-1` ve onun içinde `J ← 0 || 1 || M-1` için:
   - `tmp = resim[I][J]`
   - `counter[tmp] = counter[tmp] + 1`
5. `I ← 0 || 1 || 255` için I ve `counter[I]` yazdırılır.
6. Dur.

İç içe döngünün gövdesi önce bir öğrenciyle kuruldu: Matrisin o gözündeki değer alınır, sayaç dizisinde bu değerin gözü bir artırılır. Hoca değeri açık olsun diye `tmp`'ye aldı.

Son döngüde yalnız `counter[I]` yazmak yetmez. Şu renkten şu kadar var diyebilmek için renk bilgisi I da birlikte yazdırılmalıdır.

Hoca akış diyagramında 0'dan 255'e giden döngüyü yazarken bir yorumu da yanıtladı. Akış diyagramında döngülerin 1'den başlaması zorunlu değildir. Hoca anlatımı kolaylaştırmak için 1'den başlatıyor, ama doğru çizildiği sürece 0'dan başlayıp $N-1$'de bitebilir.

Sayaç dizisinin boyutu, saymak istediğimiz farklı değerlerin sayısına bağlıdır. Hoca bir metindeki harflerin histogramı için kaçlık dizi gerektiğini sordu. Bir öğrenci Türkçe karakterleri

düşünerek 29 dedi. İngilizce harfler için cevap 26'dır: Sıfırıncı göz A'ya, 26. göz Z'ye karşılık gelir.

> **Ders sonrası not:** Hoca sıfırıncı gözden sonra "26. göz" dedi; 0'dan başlayan numaralamada Z'nin gözü büyük olasılıkla 25 numaralı gözdür. "26." ifadesi büyük olasılıkla sıra sayısı anlamında söylenmiştir.

Soru: Alfabe için sayaç dizisinin elemanlarını nasıl belirleyeceğiz? **Cevap:** İlk gözün A'ya, son gözün Z'ye denk geldiğini bilmek yeterlidir. Böylece kaçıncı harfin hangi göze denk geldiği bilinir. Burada 0-255 değerleri gözlere doğrudan denk gelir. Değerler 1'den 256'ya kadar olsaydı $I - 1$ kullanılacaktı.

4.5 Yanlış Değil Ama Pahalı: Matrisi 256 Kez Gezmek

Bir öğrenci, iki matris döngüsünü 0'dan 255'e giden döngünün içine koyduğunu anlattı. Her renk için sayacı sıfırlıyor, matrisi baştan gezerek o rengi sayıyor ve sonucu yazıyordu.

Hoca bu çözümün dezavantajını sordu. Öğrencilerden "bir for fazla" ve "olmayan renkler için de dönüyoruz" cevapları geldi. Genel sorun şudur: Matris 256 kez gezilir. Sonuç doğrudur, çözüm yanlış değildir. Ama istenen, matrisi bir kez gezerek bütün renklerin sayısını bulmaktır.

> **Hocanın vurgusu:** Burası çok önemlidir. Önce sayaç dizisi bir kez sıfırlanır. İkinci adımda matris bir kez gezilir ve her renkten kaç tane olduğu bulunur.

4.6 Az Sayıda Renk Varsa: İkiye K'lık Tablo

Bir yorum, sayısı sıfır olan renklerin yazdırılmayabileceğiydi. Bu, önemli bir noktayı açtı. 256'lık dizi açıldığında resimde o kadar çeşit renk olmayabilir. Örneğin çoğu 0 ve 255 olan, arada yalnızca birkaç 100, 120, 130 değeri bulunan bir resim düşünün. Geri kalan gözlerin hepsi 0 kalır. Yani 4-5 renk için 256'lık alan işgal edilir.

Bellek kısıtı varsa bu daha da önemlidir. 24 bitlik bir resimde 256 yetmez, 2^{24}'lük alan açmak gerekir. Bu renklerin hepsi resimde yoksa böyle bir alan açmak çok anlamsızdır.

Bir öğrenci, yeni bir renkle karşılaşıldıkça diziye yeni eleman eklemeyi önerdi. Hoca bu fikri ikiye K'lık bir matrisle gösterdi. Bir satırda renk, ötekinde sayısı tutulur. Değerler `0, 5, 0, 10, 0` sırasıyla gelsin:

Gelen değer	Tabloda var mı?	İşlem	Tablo (renk: sayı)
0	Yok	Eklenir, sayısı 1	0: 1
5	Yok	Eklenir, sayısı 1	0: 1, 5: 1
0	Var	Sayısı 2 olur	0: 2, 5: 1
10	Yok	Eklenir, sayısı 1	0: 2, 5: 1, 10: 1
0	Var	Sayısı 3 olur	0: 3, 5: 1, 10: 1

Yeni bir değer geldiğinde önce tablodaki renkler kontrol edilir. Renk varsa sayısı bir artırılır, yoksa eklenir ve sayısı 1 olur. Sonunda yalnızca resimde geçen renkler için, renk sayısı çarpı iki kadar alan kullanılmış olur.

Soru: Akış diyagramında I'yı 0'dan başlatabildiğimizi anladım. Peki başlangıçtaki `N, M, resim[N][M]` ifadesi 0'dan mı 1'den mi başladığını belirtiyor mu? **Cevap:** Hayır. Bu ifade yalnızca N satırlı, M sütunlu bir matrisin okunacağını söyler, bir tanımlama değildir. Okuma for ile açıkça yazılsaydı, burada $N - 1$ ve $M - 1$ kullanıldığı için döngüler de `0..N-1` ve `0..M-1` olmalıydı.

Soru: C'de dizi tanımlarken bir boyut vermek gerekiyor. O değer aşılırsa ne olur? **Cevap:** Şu anda statik tanımlama yapıyoruz. Bellekte o kadar yer program çalışmadan önce ayrılır. Bu yüzden genelde kullanılacak miktarın üzerinde bir değer girilir: 100, 300, 500 gibi. İleride dinamik bellek ayırma görülecektir. Yeni eleman geldikçe dizinin boyutu bir büyütülür, böylece baştan bir boyut vermek gerekmez.

> **Hocanın vurgusu:** İkinci sınıfa kadar dizi ve matrisleri hep sabit sayılarla tanımlayın. Örneğin C'de `resim[100][100]` yazın, hiçbir zaman `resim[N][M]` yazmayın. Akış diyagramındaki `resim[N][M]` kesinlikle bir tanımlama değil, okuma yapıldığını gösteren bir ifadedir. Bazı derleyiciler `int resim[N][M]` yazımını hata saymaz, ama her C derleyicisinde çalışması için statik tanımda sabit değer yazılmalıdır.

Hoca bu kuralı ANSI C standardına dayandırdı ve ANSI'yi "American National Standard Institute" diye açıkladı.

> **Ders sonrası not:** Hoca kurumun adını "American National Standard Institute" diye söyledi; doğrusu büyük olasılıkla "American National Standards Institute"tur.

> **Ek bilgi (derste anlatılmadı):** `int resim[N][M]` yazımının bazı derleyicilerde çalışması büyük olasılıkla C99 standardıyla gelen değişken uzunluklu dizilerden (variable length array) kaynaklanır. Bu özellik büyük olasılıkla C11'de isteğe bağlı hâle getirilmiştir.

4.7 C Kodu

Akış diyagramının C karşılığı aşağıdadır. Matris hocanın istediği gibi sabit boyutla tanımlanmıştır.

```c
#include <stdio.h>

int main(void)
{
    int resim[100][100];
    int counter[256];
    int N, M, i, j, tmp;

    scanf("%d %d", &N, &M);
    for (i = 0; i < N; i++)
        for (j = 0; j < M; j++)
            scanf("%d", &resim[i][j]);

    for (i = 0; i <= 255; i++)
        counter[i] = 0;

    for (i = 0; i < N; i++)
        for (j = 0; j < M; j++) {
            tmp = resim[i][j];
            counter[tmp] = counter[tmp] + 1;
        }

    for (i = 0; i <= 255; i++)
        printf("%d %d\n", i, counter[i]);
    return 0;
}
```

Kod `gcc -Wall` ile uyarısız derlenmiştir. Dersteki `0 5 0 10 0` değerleri tek satırlık matris olarak verildiğinde sıfırdan farklı sayaçlar 0 için 3, 5 için 1 ve 10 için 1 çıkmıştır.

5. Kaydırmalı Şifreleme ve Modüler Aritmetik

Hoca, öğrencilerin daha önce üzerinde çalıştığı şifreleme (encryption) ve deşifreleme (decryption) problemine döndü. Metindeki her harf belli bir miktar kaydırılır. Bunun için iki bilgi gerekir: ne kadar kaydırılacağı (X) ve kaydırmanın sağa mı sola mı yapılacağı (R, L).

Örneğin `ABCD`, 2 sağa kaydırılsın. Sağa kaydırma artı demektir: A, C olur; B, D olur; C, E olur; D, F olur. Sonuç `CDEF`'tir. Deşifreleme de ters yönde yapılır.

Hoca bunun bilinen çok eski şifreleme yöntemlerinden biri olduğunu söyledi. Artık pek kullanılmaz, çünkü klasik bir yöntemdir ve bir örüntü izlenerek rahatça çözülebilir.

Ek bilgi (derste anlatılmadı): Bu yöntem büyük olasılıkla Sezar şifresi (Caesar cipher) adıyla bilinen yöntemdir. Hoca dersin bu kısmında bu adı kullanmadı.

5.1 Sorun: Alfabenin Sonu

Asıl zor nokta alfabenin sonudur. `Z Y X` metnini 1 sağa kaydırmak istediğimizi düşünün. Z artı 1, A'ya dönmelidir. Y, Z olur; X, Y olur. Sonuç `AZY`'dir. (ara adım eklendi)

Bir öğrenci sınırları kontrol ettiğini anlattı. Sınır aşılırsa 27 gibi bir sayı eklediğini ve X'in 27'ye göre modunu aldığını hatırlıyordu, ama sınırları tam hatırlamadığını da söyledi. Sınıfın sonra kullandığı değer 26'dır. Başka bir öğrenci 26 çıkardığını söyledi.

Hoca bunun tek başına olmayacağını gösterdi: Z'de değil de A'daysam 26 çıkarınca nereye giderim? Öğrenci bunun için ayrı koşullar koyduğunu söyledi. Bu da üretilebilir bir çözümdür. Hocanın sorusu, kontrol yerine modüler aritmetik kullanıp kullanamayacağımızdı.

Bir öğrenci X'in kendisi için mod kullandığını söyledi, çünkü X 50 gibi bir sayı olabilir. Bu birinci önemli noktadır. X, 26'dan büyükse birden fazla tur atılır, oysa kayma alfabenin içinde kalmalıdır. X = 50 için $50 \bmod 26 = 24$ olur, yani bir tam tur atılıp 24 kaydırılmış olur. (ara adım eklendi)

5.2 Mod Nereye Yazılmalı?

Sınıfta sırayla şu formüller tartışıldı.

`(metin[i] + X) mod 26`. Hoca bu formülü sınıfa sordu. Bir öğrenci olmayacağını söyledi: ASCII tablosunda karakterlerin yerleri değişmiş olur. Mod, harfin kodundan değil, A'ya göre farkından alınmalıdır.

`(metin[i] - 'A' + X + 26) mod 26 + 'A'`. Bir öğrenci bunu önerdi. Önce harfin A'dan farkı bulunur, X eklenir, 26 eklenir, mod 26 alınır ve A geri eklenir. 26'yı, A'dan eksiye doğru gidilirse sorun çıkmasın diye eklemişti.

`metin[i] + (metin[i] - 'A' + X) mod 26`. Başka bir öğrenci, ifadeyi sadeleştirmek için sondaki A'yı ve 26'yı silip başa `metin[i]` eklemeyi önerdi.

TMP ile if kontrolü. Bir başka öğrenci `metin[i] - 'A' + X` değerini bir değişkende sakladığını, 0 ile 25 arasındaysa doğrudan kullandığını, 25'ten büyükse modunu aldığını söyledi. Hoca bu kontrole gerek olmadığını, doğrudan mod alınabileceğini söyledi.

5.3 Formülleri Sınamak

Hoca son gelinen `(metin[i] - 'A' + X) mod 26` ifadesini denedi. Şimdilik yalnız büyük harfleri düşünelim, ama alfabede büyük ve küçük harf olma durumu da değerlendirilmelidir.

Girdi A, $X = 2$ olsun. A'dan A çıkınca 0 kalır, X ile 2 olur, $2 \bmod 26 = 2$ bulunur. Elimizde yalnızca 2 vardır, yani bir şey eksiktir. Bir öğrenci başa `metin[i]` eklenmesini önerdi. Bu durumda A + 2 = C olur ve doğru görünür.

Girdi Z olsun:

- $Z - A = 25$
- $25 + 2 = 27$
- $27 \bmod 26 = 1$
- `metin[i]`, yani Z'ye 1 eklenince sonuç Z + 1 olur.

Bu değer alfabenin dışındadır. Yani formül A için tesadüfen doğru çıkar, Z için yanlıştır. Bir öğrenci kendi koduna bakıp doğru düzeltmeyi söyledi: `metin[i]` yerine A eklenmelidir.

$$(\text{metin}[i] - \text{'A'} + X) \bmod 26 + \text{'A'}$$

Yeniden denersek: $25 + 2 = 27$, $27 \bmod 26 = 1$, A + 1 = B. Sonuç doğrudur.

Bu, ikinci formüle çok yakındır. Oradaki +26'nın görevi sola kaydırmadır. Öğrencinin açıklamasına göre A'dan sola kaydırınca farkı eksi bir sayı çıkar. Üzerine 26 eklenip mod 26 alınınca

sola kaydırma doğru sonuç verir. Hoca, başta artı kaydırma için çözüm arandığını, eksi için de uygulanabilmesi istenince bu eklemenin sorunsuz olduğunu söyledi.

Formül	Sonuç
`(metin[i] + X) mod 26`	Yanlış. Harfin kodunun modu alınır.
`metin[i] + (metin[i] - 'A' + X) mod 26`	Yanlış. A için doğru görünür, Z için alfabenin dışına çıkar.
`(metin[i] - 'A' + X) mod 26 + 'A'`	Sağa kaydırma için doğru.
`(metin[i] - 'A' + X + 26) mod 26 + 'A'`	Sola kaydırmayı da karşılar.

> **Ek bilgi (derste anlatılmadı):** C'de `%` işlecinin sonucu büyük olasılıkla bölünen sayının işaretini alır. Nitekim gcc ile denendiğinde `-1 % 26` değeri `-1` çıkmıştır. Bu yüzden +26 eklemesi C kodunda gereklidir. X'in mutlak değeri 26'yı aşabiliyorsa +26 tek başına büyük olasılıkla yetmez, X önce mod 26'ya indirilmelidir.
>
> **Hocanın vurgusu:** Asıl vurgulanmak istenen nokta, modüler aritmetiğin önemidir. Baştan sona dönmeyi gerektiren döngüsel (circular) yapılarda modüler aritmetik kullanılmalıdır. Kontrol yapmak yerine bunu tercih edebilirsiniz, çünkü işi daha kolay anlaşılır ve kurgulanır hâle getirir.

Hoca farklı çözümlerin de olabileceğini ve sorun olmadığını belirtti.

5.4 C Kodu

Aşağıdaki kod, büyük harfler için bu formülü kullanır. Sınıfta söz edildiği gibi önce X'in modu alınır, sonra +26 eklenir.

```c
#include <stdio.h>

char kaydir(char c, int X)   /* X > 0 sağa, X < 0 sola; c büyük harf */
{
    X = X % 26;
    return (c - 'A' + X + 26) % 26 + 'A';
}

int main(void)
{
    char metin[100] = "ABCD";
    int i;

    for (i = 0; metin[i] != '\0'; i++)
        metin[i] = kaydir(metin[i], 2);
    printf("%s\n", metin);

    for (i = 0; metin[i] != '\0'; i++)
        metin[i] = kaydir(metin[i], -2);
    printf("%s\n", metin);

    printf("%c\n", kaydir('Z', 2));
    return 0;
}
```

Kod `gcc -Wall` ile uyarısız derlenmiş, sırasıyla `CDEF`, `ABCD` ve `B` çıktılarını vermiştir.

6. Mod mu, If mi?

Dersin son bölümünün başında bir öğrenci aynı konuya döndü.

Soru: Her adımda mod alıyoruz ve araştırdığım kadarıyla mod maliyetli bir işlem. Ben X'i en başta 0-26 aralığına çektim, sonra mod almak yerine sonucun sınırı geçip geçmediğini if ile

kontrol ettim. Böyle daha az işlem mi yaparım? **Cevap:** Bu gerçekten önemli bir tartışma konusudur. Modüler aritmetikte bölme, çarpma ve çıkarma vardır, bölme ve çarpma az maliyetli işlemler değildir. If'te de bir kontrol işlemi yapılır. If'in daha az maliyetli olduğu durumlar olabilir. Bu yüzden konu probleme özgü değerlendirilmelidir.

If ile modun karşılaştırılmasında bir tarafın bariz üstün olduğunu söylemek net bir cevap olmaz. İkisi de kabul edilebilir yaklaşımlardır ve mimariye bağlı olarak performanslı çalışabilir. Bakılması gereken, modüler aritmetiğin işlemci mimarisindeki komut karşılığı ile if'in oluşturduğu yapıdır. Hoca Intel, ARM ve AMD işlemci mimarilerini örnek verdi.

> **Ders sonrası not:** Hoca if'in oluşturduğu yapı için "unconditional jumping" dedi; if büyük olasılıkla koşullu atlama (conditional jump) ile gerçekleşir.

Günün sonunda hedef, işlemleri olabildiğince basitleştirmek ve kontrol sayısını azaltmaktır. Mümkün olan en az if ve en az modüler aritmetik işlemi kullanılmalıdır. Gereksiz kontrol yapıp yapmadığınıza bakılmalıdır. Hoca bu konunun alt seviye programlamada assembly diliyle ve 8086 işlemci mimarisinin komut setleriyle yerine oturacağını söyledi.

7. İki Sıralı Diziyi Birleştirme

Son konu, merge sort'un birleştirme (merging) adımıdır. Hoca merge sort'un tamamını değil, yalnızca bu adımı anlattı.

İki sıralı dizi verilsin: `1 3 5 8 9` ve `2 4 6 7 11`. İstenen, bu iki diziyi yeni ve tek bir sıralı dizide birleştiren algoritmadır. Sonuç `1 2 3 4 5 6 7 8 9 11` olmalıdır. Amaç önce bunu yapmak, sonra verimli yapmaktır. Hoca öğrencilere önce kağıt üzerinde elle denemelerini ve fikir üretmelerini söyledi.

7.1 İlk Akla Gelen Çözümler

Bir öğrenci iki fikir söyledi. Birincisi, dizilerden birini diğerinin sonuna ekleyip herhangi bir sıralama algoritması kullanmaktır. Öğrenci bunun çok verimli olmadığını da belirtti.

Hoca bu fikri en azından düşünce olarak değerli buldu, ama kaçınılması gerektiğini söyledi. Sıralama yapıldığında en iyi karmaşıklık $N \log N$'dir, ortalama bir sıralama algoritmasının karmaşıklığı ise N^2'dir. Bu yüzden iki diziyi arka arkaya ekleyip sıralamak oldukça maliyetlidir.

> **Ders sonrası not:** Hoca sıralamada en iyi karmaşıklığın $N \log N$ olduğunu söyledi. Aynı derste insertion sort'un en iyi durumda $N - 1$ adım yaptığını gördük. Burada büyük olasılıkla, sıralılığı bilinmeyen genel bir dizi için en hızlı algoritmalar kastedilmiştir.

İkinci fikir, bir diziyi ana dizi sayıp ötekinin elemanlarını tek tek alarak ana dizide yerini aramaktır. Örneğin 1 alınır, 2 ile karşılaştırılıp başa konur. Sonra 3 alınır, 2 ile 4 arasına yerleşir.

7.2 İki İndisli Çözüm

Başka bir öğrenci aranan çözümü söyledi. Birinci dizide I, ikinci dizide J indisi tutulur. Her adımda `Dizi1[I]` ile `Dizi2[J]` karşılaştırılır. Küçük olan yeni diziye yazılır ve yalnızca onun indisi bir artırılır.

Adım	Karşılaştırma	Yazılan	Artan indis
1	1 ile 2	1	I
2	3 ile 2	2	J
3	3 ile 4	3	I
4	5 ile 4	4	J
5	5 ile 6	5	I
6	8 ile 6	6	J
7	8 ile 7	7	J
8	8 ile 11	8	I
9	9 ile 11	9	I

Bu noktada birinci dizi biter. İkinci dizide kalan 11 sona yazılır.

Hoca bu işin kaç adımda bittiğini sordu ve bir öğrencinin cevabını onayladı: $N + M$ adım. Az önce konuşulan N^2 ve $N \log N$ değerlerine göre burada iş yalnızca iki dizinin boyutu kadar sürer.

> **Ders sonrası not:** Hoca karşılaştırma sayısının $N+M$ olduğunu söyledi. Yukarıdaki örnekte $N + M = 10$ eleman yazılır, ama karşılaştırma sayısı 9'dur. $N + M$ yazma adımı için kesindir. Karşılaştırma sayısı ise büyük olasılıkla en fazla $N + M - 1$ olur. Bu fark karmaşıklığın mertebesini değiştirmez.

7.3 Özel Durum: Dizilerden Biri Erken Biterse

Hoca başka hangi durumlara dikkat etmek gerektiğini sordu.

Soru: Dizilerden biri sıralı değilse aynı yöntem kullanılabilir mi, yalnızca daha uzun mu sürer?
Cevap: Burada diziler sıralı veriliyor. Sıralı olmasalardı, örneğin `5 8 3` gibi bir dizide sorun çıkardı. Çünkü geri gelme ihtimali yoktur.

Bir öğrenci önemli bir durumu söyledi: Bir dizinin bütün elemanları diğerininkilerden küçükse ne olur? Hoca `1 3 5 7` ve `8 9 10 11 12` dizilerini yazdı. 1, 3, 5 ve 7 sırayla 8 ile karşılaştırılır ve hepsi yazılır. Sonra I dizinin sınırını geçer. Karşılaştırılacak eleman kalmamıştır.

Çözüm öğrenciden geldi: Dizi sıralı olduğu için kalan elemanlar doğrudan eklenir. Yani dizilerden birinin sonuna gelindiyse diğer dizi, nerede kaldıysa oradan itibaren yeni dizinin sonuna tek tek eklenir. Bu örnekte sonuç `1 3 5 7 8 9 10 11 12` olur.

Soru: Dizilerin uzunlukları farklıysa sorun çıkar mı? I sona geldiğinde J'de bir sorun olur mu?
Cevap: Olmaz. I'nın sona gelip gelmediği dizinin uzunluğu olan N ile, J'ninki M ile kontrol edilir. I, N'den ve J, M'den küçük kaldığı sürece devam edilir. Sınır aşılınca o dizinin bittiği anlaşılır ve diğer dizi kaldığı yerden sona yazılır.

Algoritmanın sözle anlatılan adımları şöyledir:

1. I, J ve yeni dizinin indisi başlangıca alınır.
2. `I < N` ve `J < M` olduğu sürece küçük olan eleman yeni diziye yazılır, o dizinin indisi ve yeni dizinin indisi artırılır.
3. Birinci dizide eleman kaldıysa kalanlar yeni dizinin sonuna yazılır.
4. İkinci dizide eleman kaldıysa kalanlar yeni dizinin sonuna yazılır.

Bu algoritmanın akış diyagramı derste çizilmedi, öğrencilere çalışma sorusu olarak bırakıldı. Aşağıdaki kod bu adımların C karşılığıdır.

```c
#include <stdio.h>

int main(void)
{
    int a[100] = {1, 3, 5, 8, 9};
    int b[100] = {2, 4, 6, 7, 11};
    int c[200];
    int N = 5, M = 5;
    int i = 0, j = 0, k = 0;

    while (i < N && j < M) {
        if (a[i] < b[j]) {
            c[k] = a[i];
            i = i + 1;
        } else {
            c[k] = b[j];
            j = j + 1;
        }
        k = k + 1;
    }
    while (i < N) {
        c[k] = a[i];
```

```
        i = i + 1;
        k = k + 1;
    }
    while (j < M) {
        c[k] = b[j];
        j = j + 1;
        k = k + 1;
    }

    for (i = 0; i < k; i++)
        printf("%d ", c[i]);
    printf("\n");
    return 0;
}
```

Kod `gcc -Wall` ile uyarısız derlenmiş ve `1 2 3 4 5 6 7 8 9 11` çıktısını vermiştir.

7.4 Alternatif: Dizilerin Sonuna Büyük Bir Değer Eklemek

Bir öğrenci alternatif önerdi: İki dizinin sonuna çok büyük bir değer eklemek. Hoca bunun da uygulanan yöntemlerden biri olduğunu söyledi. Dizilerin sonuna örneğin 1000 gibi bir sayı eklenerek dizilerin hiç bitmemesi sağlanır.

Hoca `1 3 5 7` ve `8 9 10 11 12` örneğinde her iki dizinin sonuna 13 koymayı düşündü. Eklenen değer, dizilerin son elemanlarından büyük olmalıdır. Birinci dizinin sonuna 8 konamaz, çünkü 7'nin bir büyüğü diğer dizinin elemanlarından büyük olmaz. Hoca bu çözümün üzerinde düşünülmesini istedi.

Dersin Toparlaması

Bu derste insertion sort'un akış diyagramı, hatalı denemeler üzerinden adım adım kuruldu. `Dizi[I]`'nın kaydırma sırasında kaybolması TMP'yi, her turdaki kaydırma sayısının belli olmaması while kullanımını gerektirdi. Sıralı bir dizide $N - 1$ karşılaştırma yapıldığı görüldü, bu da algoritmanın sıralıya yakın veride neden avantajlı olduğunu açıkladı. Sıralılığı bilinmeyen büyük dizilerde ise merge sort ve quick sort gibi algoritmalar tercih edilir.

Sonraki örnekler üç algoritmik bakış açısı verdi. Histogramda çok sayıda sayaç bir dizide toplandı ve matris yalnızca bir kez gezildi. Şifrelemede döngüsel yapı modüler aritmetikle çözüldü. İki sıralı diziyi birleştirmede iki indisle $N + M$ adımda sonuca varıldı. Birleştirmenin akış diyagramı öğrencilere bırakıldı. Hoca önümüzdeki hafta muhtemelen işletim sistemi ve ağ konularının işleneceğini, bir hafta sonra algoritmik problemlere dönüleceğini söyledi.

Not: İsterseniz bu kayıtları NotebookLM gibi bir araca atıp daha istifadeli hale getirebilirsiniz.